Group: Manav Sainani & Christa-Marie Seerattan

Summary of Results:

Our version of PacMan includes the implementation of 6 ghosts, utilizing various targeting logic approaches based on defined algorithms and heuristics:

Ghost Name		Algorithm	Behavioral Modifier	Targeting Logic
Original	Variation			
Blinky	**RedGhost**	Breadth First Search (BFS)	-	**Direct Ambush** PacMan's current tile
Clyde	**OrangeGhost**	A*	Distance Threshold	**Chase & Scatter** Distance > 8 tiles: chase Distance <= 8 tiles: flee
Pinky	**PinkGhost**		Directional Prediction	**Anticipation** Predict PacMan's moves (2 tiles ahead)
Funky	**GreenGhost**		Detection Range	**Need for Speed** Distance >12 tiles: random Distance <= 12 tiles: speed boost
Inky	**BlueGhost**	A* & Breadth First Search Hybrid	Algorithm Toggle	**Switch & Conquer** Alternates between PacMan's current and future positions
Sue	**PurpleGhost**	Depth Limited Search (DLS)	Depth Limit	**Meet the Quota** Reachable within 12 tiles: chase Not reachable within 12 tiles: random

Breadth First Search (BFS) is a graph traversal algorithm that explores a graph level by level. Using a queue (FIFO), all of the neighbors of a given node are visited before traversing deeper into the graph. As a result, BFS finds the shortest path in an unweighted graph, where each edge carries equal cost and thereby the graphs are explored uniformly. PacMan's maze utilizes the same traversal method, where movement along the tiles are also of equal cost. BFS is used to find the shortest path overall to PacMan, if we ignore the assumptions of its direction. This algorithm, used for Blinky, is based on a reactive targeting protocol which gets the optimal distance to PacMan. The consequence of this algorithm is that all nodes are explored which is not the most time efficient approach.

A* Algorithm introduces a heuristic which rectifies the limits of the BFS. With each step towards the target, it calculates an estimate of the remaining cost and makes the best decision. Instead of treating all neighbors equally, A* evaluates the best path to the target by using the equation $f(n) = g(n) + h(n)$, where n is the specific node, $f(n)$ is the estimated total cost to reach goal through n, $g(n)$ is the accumulated cost from start node to n, and $h(n)$ is the heuristic estimate to the destination. For PacMan's maze, we utilize the Manhattan distance (shortest distance using sum of absolute differences of the coordinates) which gives the exact estimate. As a result, A* finds the optimal path but unlike BFS, it explores fewer tiles and is more time efficient. The consequence of this algorithm is that its performance depends entirely on the quality of the heuristic chosen. This means that different behaviors can arise from the same algorithm:

- Clyde always chases but switches what it chases based on distance, staying active but unpredictable.
- Pinky always chases and always aims ahead of PacMan, staying active and predictive.
- Funky switches between chasing and not using A* at all based on distance. Unlike Clyde, Funky does not change its target but changes whether or not the algorithm runs at all, staying conditionally active and reactive when triggered.

Depth Limited Search (DLS) is a modified Depth First Search (DFS) algorithm. DFS is a graph traversal algorithm that explores a graph to its furthest level and backtracks (recursively in our implementation) to find the desired target node. With DLS, the graph is only traversed up until a specified depth limit. This algorithmic approach is more memory efficient and prevents infinite loops as it only keeps track of the branch being explored, when compared to BFS and A* which keep track of all possible paths. The tradeoff of this algorithm is completeness as if PacMan is located beyond the depth limit, Sue abandons the search entirely and falls back to random movement. This strategy is easily exploitable by simply keeping distance.

Based on the aforementioned, it can be concluded that both BFS and A* have strengths and weaknesses as it relates to grid navigation. BFS works better for short-range pursuit where the target is nearby and the search space is small, since it guarantees the shortest path without relying on any heuristic. A* works better for long-range pursuit and larger search spaces, where the heuristic allows it to have a focused search toward the target and avoids unnecessary traversals, making it more time efficient. Like BFS, DLS also works better for short-range pursuit and small search space, but does not guarantee the shortest path (or any path) to the target.

The hybrid ghost, Inky, allows us to exploit the advantages of both the BFS and A* algorithms. On alternating ticks, it switches between the BFS reactive strategy and A*'s predictive strategy. By toggling between the two every update frame, Inky exhibits both behaviors in rapid succession, making him harder to evade than either algorithm alone. This was done by flipping a boolean to run either BFS or A*. We decided to employ Inky this way rather than merging in order to efficiently and effectively run both algorithms while avoiding the complexity of conflicting path outputs. The algorithm oscillation allows us to achieve the hybridization goal without compromising on the quality or run time of the algorithms. Since Inky chases and intercepts, its behavior is not consistent and therefore harder to evade.

Based on the underlying pursuit-evasion problem, a hybrid algorithm is more effective than BFS or A* on their own. In the paper, the issue being addressed revolved around the behavior of pursuers against a target with agency, specifically, how a swarm of UAVs can coordinate to intercept an unpredictable, evasive target. As highlighted, this challenge is prevalent in real world applications such as robotics, autonomous systems, and adversarial AI. A purely reactive agent fails against a target that strategically evades, and a purely predictive agent fails when the target behaves unpredictably. The hybrid covers both failure points simultaneously, making it more robust than either approach alone. However, it is worth noting that hard-coded hybrids like Inky's are deterministic not dynamic, hence the need for adaptable coordination strategies based on the research paper. As seen with Funky and Sue, leaving points of susceptibility makes agents exploitable and ultimately limits their effectiveness, which is why the paper advocates for decentralized policies over well-designed, but static, algorithmic approaches.